Lecture 8 Graph Neural Networks

FTEC 6910F Machine Learning for Finance

Dr. Li. Ruikun

ruikunli@hkust-gz.edu.cn
Financial Technology Thrust, Society Hub
The Hong Kong University of Science and Technology(GZ)

March 23, 2026

1. Graph Concepts and Motivation
2. Graph Representation
3. Graph Convolution Networks
4. Graph Learning Tasks
5. Guest Speaking

1 Graph Concepts and Motivation

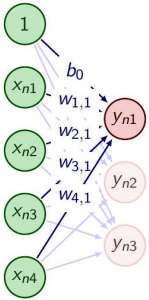
2 Graph Representation

3 Graph Convolution Networks

4 Graph Learning Tasks

5 Guest Speaking

Two Views of a Linear Layer in Deep Learning



$$\begin{pmatrix} y_{n1} \\ y_{n2} \\ y_{n3} \end{pmatrix} = \begin{pmatrix} w_{1,1} & w_{2,1} & w_{3,1} & w_{4,1} \\ w_{1,2} & w_{2,2} & w_{3,2} & w_{4,2} \\ w_{1,3} & w_{2,3} & w_{3,3} & w_{4,3} \end{pmatrix} \begin{pmatrix} x_{n1} \\ x_{n2} \\ x_{n3} \\ x_{n4} \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix}$$

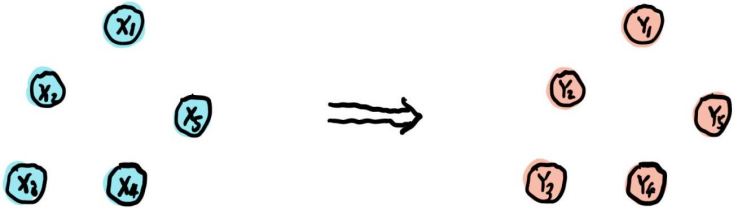
$$= \begin{pmatrix} b_1 & w_{1,1} & w_{2,1} & w_{3,1} & w_{4,1} \\ b_2 & w_{1,2} & w_{2,2} & w_{3,2} & w_{4,2} \\ b_3 & w_{1,3} & w_{2,3} & w_{3,3} & w_{4,3} \end{pmatrix} \begin{pmatrix} 1 \\ x_{n1} \\ x_{n2} \\ x_{n3} \\ x_{n4} \end{pmatrix}$$

- Individual view: data by data into a network,

$$\mathbf{x}_n \in \mathbb{R}^d \longrightarrow \mathbf{y}_n = \mathbf{W}^\top \mathbf{x}_n \in \mathbb{R}^{d_1} \qquad \text{(data in column shape)}$$

- Conventional Deep NNs: $\mathbf{y}_n = f(\mathbf{x}_n)$: Data-by-Data

Two Views of a Linear Layer



- Overall view: Taking all the data into a matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$, then

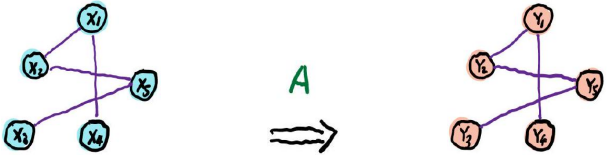
$$\mathbf{Y} = \mathbf{X}\mathbf{W} \in \mathbb{R}^{N \times d_1} \quad \text{(data in row shape)}$$

- Columns correspond to features, while rows correspond to data items. $\mathbf{W}$ is used to only mix up feature components inside each data vector.

Assumption on Data Independence

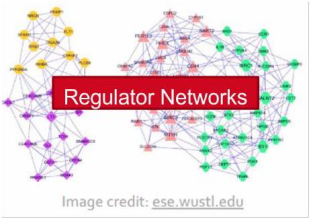
- For a given training dataset $\mathcal{D} = \{(\mathbf{x}_n, t_n)\}_{n=1}^{N}$, we train a model based on the fundamental assumption that data items are independent of each other
- Because of this assumption, we were looking at models whose output is calculated at a particular data, as $\widehat{y}_n = f(\mathbf{x}_n, \mathbf{W})$ aiming at the observed target t_n (in supervised learning scenario)
- However, in the real world, there are many scenarios where there must be a certain "relation" between data $\{\mathbf{x}_n\}$.
- How could we explore this "information" to enhance our model design and learning?

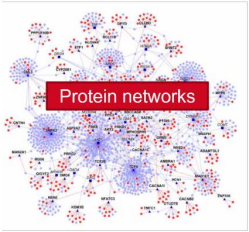
Handling "Relation Information"



- One way to encode such "relation information" is through the so-called graph object
- For example, if data $\mathbf{x}_1$ and $\mathbf{x}_2$ are related to each other, like both customers are in a similar age, at a similar pay-scale, etc., we can link them with an edge.
- A graph is a mathematical object that consists of a number of nodes (i.e. custermers or data) and a number of edges between nodes.
- The graph can perfectly define that "relation information", particularly useful in representing unstructured data

Real World Unstructured Data



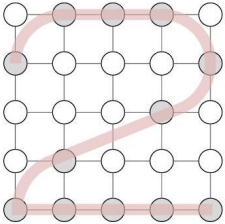


Transaction Data in Business: Defining Relations between Customers and Items

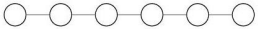
	buy_id	sell_id	volume
1	0	0	476496000
2	1	0	486432000
3	2	1	482803200
4	3	1	474422400
5	4	1	475372800
6	5	2	448848000
7	6	2	463795200
8	7	3	411264000
9	8	4	443059200
10	9	4	465696000

Graph Data

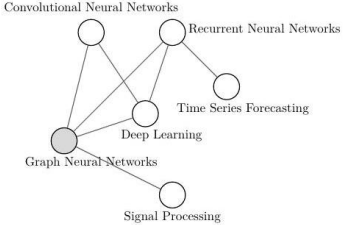
So far, we have always assumed the data are i.i.d. Clearly, real-world data do not satisfy this assumption.



(a) Grid



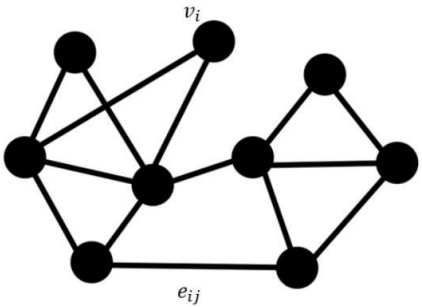
(b) Sequence



(c) Graph

Concept of Graph

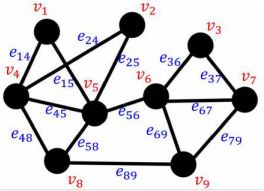
node or vertex



edge

Mathematical Concepts for Graphs

- A graph consists of nodes (vertices) and edges (or links or connections) between nodes.
- Denote an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where $\mathcal{V} = \{v_1, v_2, \ldots, v_N\}$ is a non-empty set of nodes while $\mathcal{E} \subset \mathcal{V} \times \mathcal{V}$ is a set of edges each of which e_{ij} represents a pair of node $e_{ij} = e_{ji} = \{v_i, v_j\}$.
- If each edge is ordered, $e_{ij} = (v_i, v_j)$ (meaning v_i and v_j are in order), we will call $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ a directed graph. We don't consider directed graphs.
- We denote the number of vertices and edges of the graph $\mathcal{G}$ as $|\mathcal{V}| = N$ and $|\mathcal{E}|$, respectively.



Graph Concepts: Edges

- Two nodes $v_i, v_j \in \mathcal{V}$ are said connected if their exist an edge $e_{ij} = e_{ji} \in \mathcal{E}$ between them. In this case, v_i, v_j are called one-hop neighbors to each other (see the next slide). The adjacency matrix $\mathbf{A} = [a_{ij}]$ of size $N \times N$ is defined as

$$a_{ij} = \begin{cases} 1, & \text{if } e_{ij} \in \mathcal{E}; \\ 0, & \text{otherwise} . \end{cases}$$

- Previous graph adjacency matrix

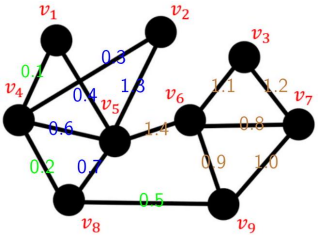
$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

Graph Concepts: Edges

When there is a weight w_{ij} on each edge e_{ij}, we can form the edge weight matrix $\mathbf{W} = [w_{ij}]$ of size $N \times N$

$$w_{ij} = \begin{cases} w_{ij}, & \text{if } e_{ij} \in \mathcal{E} \\ 0, & \text{otherwise} \end{cases}$$

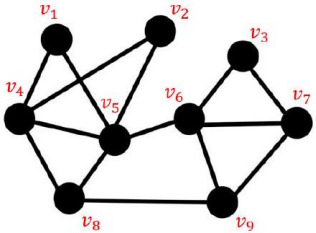
Example: Edge Weight Matrix



$$\mathbf{W} = \begin{bmatrix} 0 & 0 & 0 & 0.1 & 0.4 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.3 & 1.3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1.1 & 1.2 & 0 & 0 \\ 0.1 & 0.3 & 0 & 0 & 0.6 & 0 & 0 & 0.2 & 0 \\ 0.4 & 1.3 & 0 & 0.6 & 0 & 1.4 & 0 & 0.7 & 0 \\ 0 & 0 & 1.1 & 0 & 1.4 & 0 & 0.8 & 0 & 0.9 \\ 0 & 0 & 1.2 & 0 & 0 & 0.8 & 0 & 0 & 1.0 \\ 0 & 0 & 0 & 0.2 & 0.7 & 0 & 0 & 0 & 0.5 \\ 0 & 0 & 0 & 0 & 0 & 0.9 & 1.0 & 0.5 & 0 \end{bmatrix}$$

Graph Concepts: Node Neighborhood

- For each node v_i, we denote all the one-hop neighbors as $\mathcal{N}(v_i)$ (excluding v_i itself). The degree $\deg(v_i)$ is the number of its one-hop neighbors, i.e., $\deg(v_i) = |\mathcal{N}(v_i)|$.
- For example, for the following graph, $\mathcal{N}(v_6) = \{v_3, v_5, v_7, v_9\}$.



- 2-hop neighbors of a node v_i are defined as the set of all the one-hop neighbors and all the one-hop neighbors of each one-hop neighbor, excluding v_i itself.

Graph Concepts: Node Neighborhood

- The degree matrix (diagonal) is defined as $\mathbf{D} = \operatorname{diag}(\mathbf{A}\mathbf{1}_N)$.

$$\mathbf{D} = \operatorname{diag}\left(\underbrace{\begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}}_{\mathbf{1}_N}\right) = \begin{bmatrix} 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 4 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 5 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 3 \end{bmatrix}$$

Graph Concepts: Node Neighborhood

From the previous degree matrix, we observe that nodes with high degree become more important and have a larger scale. In this setting, what do we obtain?

- **Degree bias:** high-degree nodes produce larger scale
- Representation magnitude is dominated by the volume of edge connections, and becomes biased toward high-degree regions

The adjacency matrix $\mathbf{A}$ introduces a **degree bias**: node representations scale with degree rather than with feature semantics.

Graph Concepts: the Laplacian

To mitigate degree bias, we introduce the graph Laplacian $\mathbf{L}$ to balance degree scaling.

- The Laplacian matrix of a graph is $\mathbf{L} = \mathbf{D} - \mathbf{A}$, which is a SPD (symmetric positive semi-definite) matrix, i.e., for any $\mathbf{f} \in \mathbb{R}^N, \mathbf{f}^\top \mathbf{L}\mathbf{f} \geq 0$. The **normalized Laplacian** is $\widehat{\mathbf{L}} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$
- Sometimes people call $\mathbf{D}_w = \mathrm{diag}\left(\mathbf{W}\mathbf{1}_N\right)$ as the **weighted degree matrix**, thus the **weighted Laplacian** $\mathbf{L}_w = \mathbf{D}_w - \mathbf{W}$ which is also SPD matrix. $\mathbf{L}$ is a special case of $\mathbf{L}_w$ with all weights 1 's. The **normalized weighted Laplacian** is defined as $\widehat{\mathbf{L}}_w = \mathbf{I} - \mathbf{D}_w^{-1/2}\mathbf{W}\mathbf{D}_w^{-1/2}$

1 Graph Concepts and Motivation

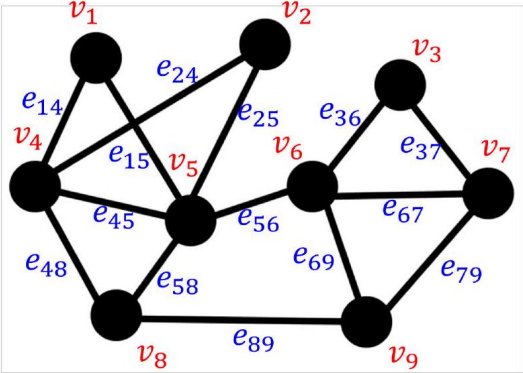
2 Graph Representation

3 Graph Convolution Networks

4 Graph Learning Tasks

5 Guest Speaking

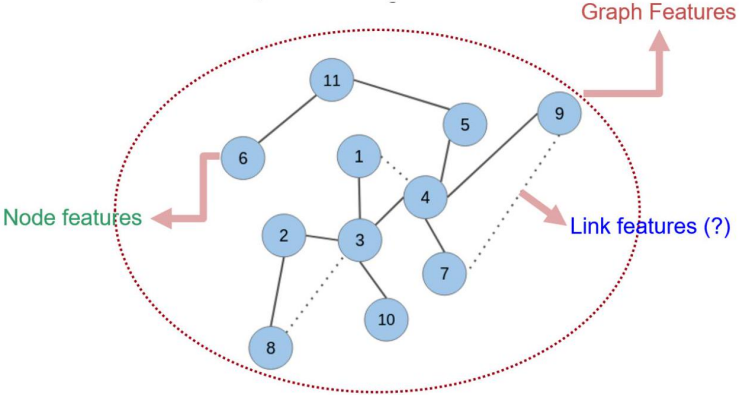
Mathematical Representation of Graphs: No Drawing is needed!



$\mathcal{G} = \{\mathcal{V}, \mathbf{A}\} \Rightarrow \mathbf{A}$ records all the edges of the graph.

Graph Features (Data)

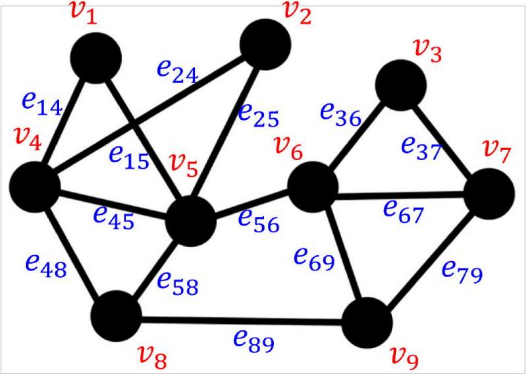
Three-Level Features on a Graph:



Graph Features (Storing Data)

- Each node represents a particular object, so the data for the object can be associated (stored) with the node
- We collect all the **Node Features** in $\mathbf{X} \in \mathbb{R}^{N \times D}$, each row of $\mathbf{X}$ corresponds to a node. **Note** that the order of rows depends on the node order. And note the order is not part of a graph. It is we who give this order.
- Edge Features will be denoted by $\mathbf{E} \in \mathbb{R}^{|\mathcal{E}| \times D_1}$, where edge features may have their own dimension D_1. Each row corresponds to an edge, and the order of rows depends on the edge order.
- For a single graph, its feature will be represented by a vector $\mathbf{g} \in \mathbb{R}^{D_2}$. For a group of Q graphs, we use $\mathbf{G} \in \mathbb{R}^{Q \times D_2}$
- Similarly, for each node (or edge), we have target data $\mathbf{t}$ (or $\mathbf{t}_e$)

Graph Data Representation



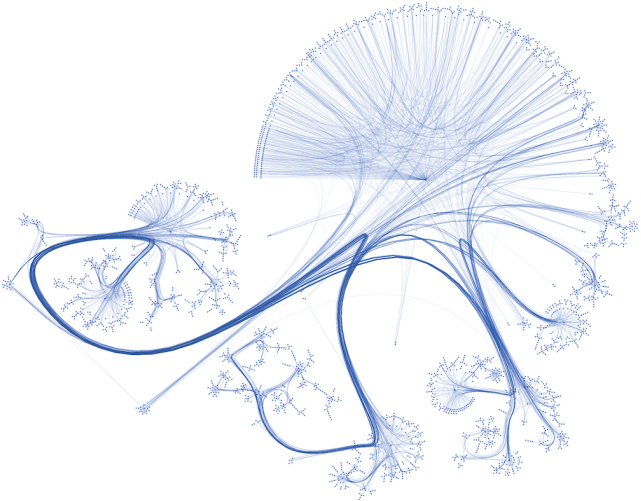
$$\Longrightarrow \quad \begin{aligned} \mathcal{G} &= \{\mathbf{X}, \mathbf{A}\} \text{ or} \\ &= \{\mathbf{X}, \mathbf{A}, \mathbf{E}\} \end{aligned}$$

$\mathbf{A}$ records all the edges of the graph.

Example: Cora Graph

- Introduction to the Cora dataset: https://relational.fit.cvut.cz/dataset/CORA
- The Cora dataset consists of 2708 scientific publications classified into one of SEVEN classes: {'Case_Based', 'Genetic_Algorithms', 'Neural_Networks', 'Probabilistic_Methods', 'Reinforcement_Learning', 'Rule_Learning', 'Theory'}
- The citation network consists of 5429 links. Each publication in the dataset is described by a 0/1-valued word vector indicating the absence/presence of the corresponding word from the dictionary. The dictionary consists of 1433 unique words.

Example: Cora Graph



1 Graph Concepts and Motivation

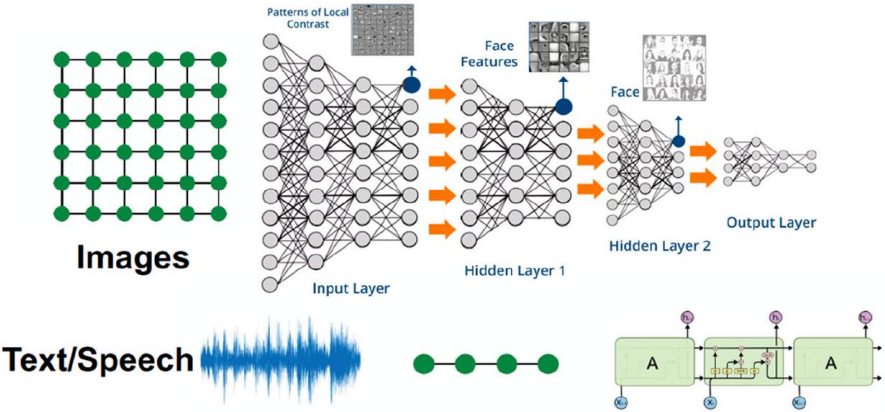
2 Graph Representation

3 Graph Convolution Networks

4 Graph Learning Tasks

5 Guest Speaking

Classic ML Packages



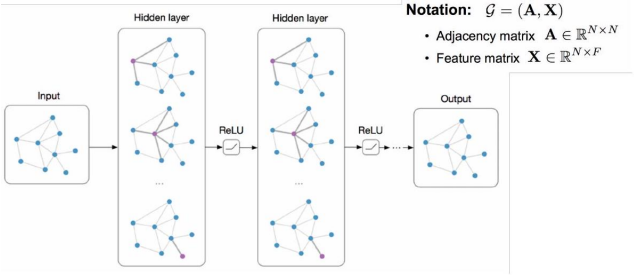
$$\mathbf{x}_n \xrightarrow{\text{a network}} \widehat{y}_n$$

How about Graph Data

The Main Question:

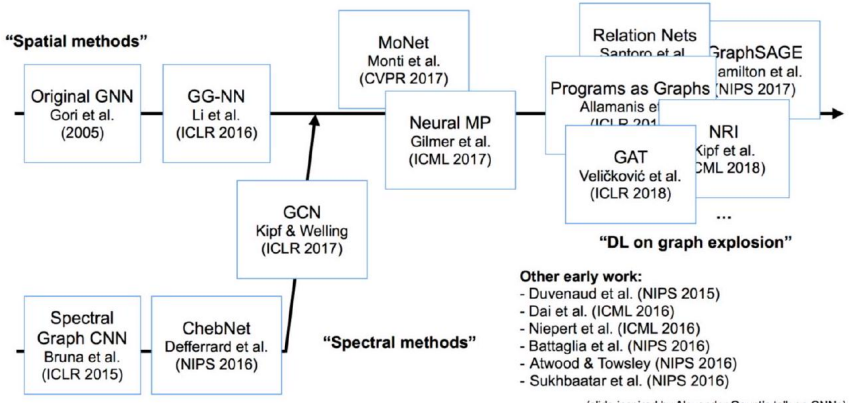
- What is the best way to design a machine learning algorithm to take advantage of relational structure of graphs?

Graph Neural Networks (GNNs)



- Main Idea: Passing messages between pairs of nodes and agglomerate
- Alternative Interpretation: Passing messages between nodes to refine node (and possibly edge) representations

A Brief History of GNNs



(slide inspired by Alexander Gaunt's talk on GNNs)

https://networkx.org/

https://pytorch-geometric.readthedocs.io/en/latest/index.html

The Basic GNN Layers

- Recall the deep networks are defined layer by layer, (to avoid confusion, we use $\mathbf{H}^{(l)}$ for the activated values $\mathbf{A}^{(l)}$),

$$\mathbf{H}^{(l)} = \sigma_l\left(\mathbf{H}^{(l-1)}\mathbf{W}^{(l)}\right), \qquad \text{for } l = 1, 2, \ldots, L.$$

  where we have removed the bias for simplicity, $\mathbf{H}^{(0)} = \mathbf{X}$ and the final model output $\mathbf{Y} = \mathbf{H}^{(L)}$.
- Without loss of generality, we consider the first layer, that is

$$\mathbf{H} = \sigma(\mathbf{X}\mathbf{W})$$

- Note that the activation function here is applied on matrix element by element. Thus the first row of $\mathbf{H}$ is calculated from the first row of $\mathbf{X}$ (the first data) and parameters $\mathbf{W}$. $\mathbf{W}$ only mixes feature components!
- For graph node feature $\mathbf{X}$, we know they are related to each other based on the graph structure. How could we incorporate this information into layer calculation?

The Basic GNN Layers

- One simple way is to define the following layer operation

$$\mathbf{H} = \sigma(\mathbf{AXW})$$

- For the graph on Slide 21, to see what is done in this layer, suppose $\mathbf{W} = \mathbf{I}$ for simplicity, we will see

$$\mathbf{AX} = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \mathbf{x}_3^T \\ \mathbf{x}_4^T \\ \mathbf{x}_5^T \\ \mathbf{x}_6^T \\ \mathbf{x}_7^T \\ \mathbf{x}_8^T \\ \mathbf{x}_9^T \end{bmatrix} = \begin{bmatrix} \mathbf{x}_4^T + \mathbf{x}_5^T \\ \mathbf{x}_4^T + \mathbf{x}_5^T \\ \mathbf{x}_6^T + \mathbf{x}_7^T \\ \mathbf{x}_1^T + \mathbf{x}_2^T + \mathbf{x}_5^T + \mathbf{x}_7^T \\ \mathbf{x}_1^T + \mathbf{x}_2^T + \mathbf{x}_4^T + \mathbf{x}_6^T + \mathbf{x}_8^T \\ \mathbf{x}_3^T + \mathbf{x}_5^T + \mathbf{x}_7^T + \mathbf{x}_9^T \\ \mathbf{x}_3^T + \mathbf{x}_6^T + \mathbf{x}_9^T \\ \mathbf{x}_4^T + \mathbf{x}_5^T + \mathbf{x}_9^T \\ \mathbf{x}_6^T + \mathbf{x}_7^T + \mathbf{x}_8^T \end{bmatrix}$$

- Here we are mixing features crossing node data!

The Basic GNN Layers

- Summary for the following simply GNN layer

$$\mathbf{H} = \sigma(\mathbf{AXW})$$

- The layer consists of three operations
  1. XW makes new features by taking the linear combination of feature components within each node feature $\mathbf{x}_n$;
  2. $\mathbf{A}(\mathbf{XW})$ takes the sum of each node's neighbors to get new features as demonstrated in the above example;
  3. Finally take whatever activation σ.
- As $\mathbf{A}$ has 0 values for its diagonal, thus a node feature was not incorporated into the new feature in $\mathbf{A}(\mathbf{XW})$. Check first two rows on the last slide.
- To keep the original feature for each node, we can add the self-loop (edge connecting the node to itself) for each node, by using

$$\mathbf{H} = \sigma(\tilde{\mathbf{A}}\mathbf{XW})$$

with $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$. Do the previous example, what has been changed?

The GCN Layer

- To avoid the issue where nodes with many neighbors become dominant, it is recommneded using the normalized adjacency matrix $\widehat{\mathbf{A}} = \widetilde{\mathbf{D}}^{-\frac{1}{2}}\widetilde{\mathbf{A}}\widetilde{\mathbf{D}}^{-\frac{1}{2}}$ to replace $\widetilde{\mathbf{A}}$, where $\widetilde{\mathbf{D}} = \operatorname{diag}\left(\widetilde{\mathbf{A}}\mathbf{1}_N\right)$.
- Finally we get the so-called GCN (graph convolution network) layer

$$\mathbf{H} = \sigma(\widehat{\mathbf{A}}\mathbf{X}\mathbf{W})$$

- Use this basic building block we can define a deep architecture for graph data as follows

$$\mathbf{H}^{(l)} = \sigma_l\left(\widehat{\mathbf{A}}\mathbf{H}^{(l-1)}\mathbf{W}^{(l)}\right), \qquad \text{for } l = 1, 2, \ldots, L.$$

  where we have removed the bias for simplicity, $\mathbf{H}^{(0)} = \mathbf{X}$ and the final model output $\mathbf{Y} = \mathbf{H}^{(L)}$.
- Now you can send $\mathbf{Y}$ to any loss function for a particular task (classification or regression).

Comparison

- The normal 2-layer neural networks (without showing $\mathbf{Z}^{(l)}$ and changing $\mathbf{A}^{(l)}$ to $\mathbf{H}^{(l)}$):

$$\mathbf{H}^{(0)} = \mathbf{X}$$
$$\mathbf{H}^{(1)} = \sigma\left(\mathbf{H}^{(0)}\mathbf{W}^{(1)}\right)$$
$$f(\mathbf{X}, \mathbf{W}) = \mathbf{H}^{(2)} = \sigma\left(\mathbf{H}^{(1)}\mathbf{W}^{(2)}\right)$$

- The basic 2-layer Graph neural networks:

$$\mathbf{H}^{(0)} = \mathbf{X}$$
$$\mathbf{H}^{(1)} = \sigma\left(\widehat{\mathbf{A}}\mathbf{H}^{(0)}\mathbf{W}^{(1)}\right)$$
$$f(\mathbf{X}, \mathbf{W}) = \mathbf{H}^{(2)} = \sigma\left(\widehat{\mathbf{A}}\mathbf{H}^{(1)}\mathbf{W}^{(2)}\right)$$

- The difference is we have added adjacency information $\mathbf{A}$ in each layer of GNN.

Implementation Issues

- It seems a single GCN layer $\mathbf{H} = \sigma(\widehat{\mathbf{A}}\mathbf{X}\mathbf{W})$ only involves matrix multiplication.
- If the number of nodes goes to tens of thousands, even for a modern computer, it is hard to fit into its memory.
- For the previous deep networks, as the calculation is done data by data, we use the batch to reduce the calculation.
- However this is not the case for GCN, as the data will be mixed according to the graph structure A. Batching GCN is an open problem
- Fortunately $\mathbf{A}$ is sparse enough, so that implementation is done by sparse matrix operation in a computer
- A possible batching way is to sample a subgraph from the given graph, however we will lose some structure information. Think about why?

Message Passing

- The GCN can be explained under the framework of message passing, a general framework to construct graph layers.
- Message Passing: The output feature representation $\mathbf{h}_i^{(l)}$ of the ith node, i.e. the ith row of the matrix $\mathbf{H}^{(l)}$, is updated by

$$\mathbf{h}_i^{(l)} = \gamma^{(l)}\left(\mathbf{h}_i^{(l-1)}, \Psi_{j\in\mathcal{N}(i)}\phi^{(l)}\left(\mathbf{h}_i^{(l-1)}, \mathbf{h}_j^{(l-1)}, \mathbf{e}_{i,j}\right)\right)$$

where Ψ denotes a differentiable, permutation invariant function, e.g., sum, mean or max, $\mathcal{N}(i)$ is the neighborhood of the i th node, $\mathbf{e}_{i,j}$ represents the feature on edge e_{ij}, and γ and ϕ denote differentiable functions such as MLPs (Multi-Layer Perceptrons). ϕ is called aggregation function while γ is called update function.

Message Passing

- The following diagram explains the GCN under the message passing.

$$\mathbf{h}_i^{(l+1)} = \sigma\left(\mathbf{h}_i^{(l)} W^{(l)} + \sum_{j\in\mathcal{N}(v_i)} \frac{1}{c_{ij}} \mathbf{h}_j^{(l)} W^{(l)}\right)$$

- The new feature $\mathbf{h}_i^{(l+1)}$ at i-th node is obtained by its own feature $\mathbf{h}_i^{(l)}$ and the features $\mathbf{h}_j^{(l)}$ of its neighbors being passed to it. And c_{ij} is a normalized constant on edge e_{ij}, fixed or learnable.

1 Graph Concepts and Motivation

2 Graph Representation

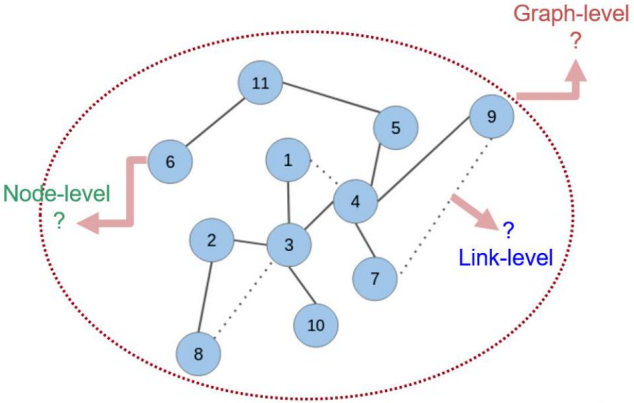
3 Graph Convolution Networks

4 Graph Learning Tasks

5 Guest Speaking

Graph Learning Tasks

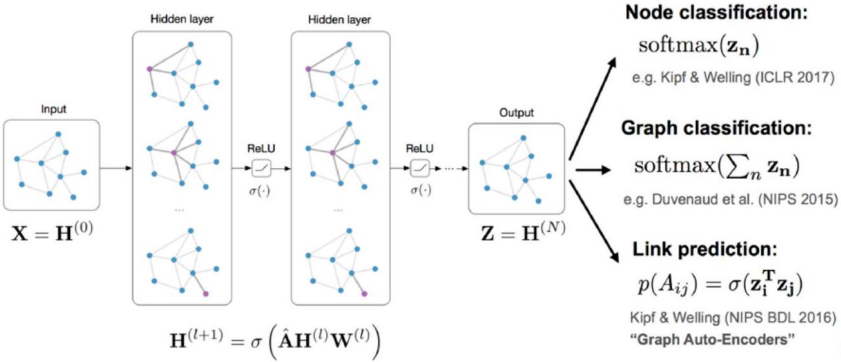
- Three levels of tasks: (1) Node-level prediction; (2) Link-level prediction; and (3) Graph-level prediction:



General Architecture for Graph Learning

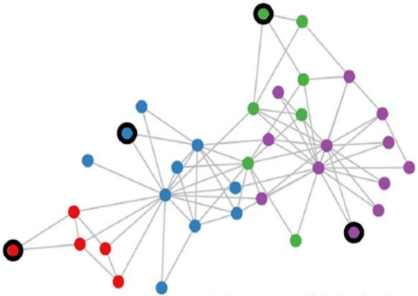
Classification and **Link Prediction** with GNNs / GCNs

Input: Feature matrix $\mathbf{X} \in \mathbb{R}^{N \times F}$, preprocessed adjacency matrix $\widehat{\mathbf{A}}$



Semi-supervised Learning (node classification)

- **Setting**: Some nodes are labelled . All other nodes are unlabeled
- **Goal**: predict all the unlabeled nodes



$$\mathcal{L} = -\sum_{n \in \mathcal{Y}_L} \sum_{k=1}^{K} Y_{nk} \log \widehat{Y}_{nk}$$

$\mathcal{Y}_L$: set of labeled node indices

$\mathbf{Y}$: the label (one-hot matrix)

$\widehat{\mathbf{Y}}$: the softmax of GCN outputs $\mathbf{H}^{(L)}$

1 Graph Concepts and Motivation

2 Graph Representation

3 Graph Convolution Networks

4 Graph Learning Tasks

5 Guest Speaking

Graph Application on Superannuation Fund

Michael Mi

deputy chief investment officer, NGS Super

Bio

Michael Mi is the deputy chief investment officer at NGS Super, an Australian-based industry superannuation fund. Formerly he was the head of portfolio construction and quantitative research. He oversees the total portfolio construction function, covering in-house strategic, dynamic, and tactical asset allocations. He also leads the quantitative research function to develop systematic strategy, build proprietary signals and quantitative tools, and support the broader fund's research agenda. Michael is a PhD in Finance from the University of Sydney, focusing on asset pricing and nonlinearity in the financial markets. He is also a CFA Charter holder.